

Smart Vocabulary

포트폴리오 설명서

1. 개요 (Overview)

Smart Vocabulary 는 현대적인 데스크탑 환경에서 AI 기술과 클라우드 동기화를 결합하여 사용자의 효율적인 외국어 학습을 돕는 스마트 영단어 암기장 애플리케이션입니다.

2. 개발 동기

영단어 암기를 위해 www.cram.com(cram)이라는 사이트를 이용하다가 불편함을 느껴서 개발하게 되었습니다.

2. 1. 단어 뜻의 자동 채우기 기능

1. 단어 카드셋에 영단어를 추가할 때마다 품사별 단어 뜻, 예문, 예문 해석, 발음 기호 등을 작성.

2. 추가할 단어가 많을수록 이 작업이 번거롭고 많은 시간이 소요됨.

=> 단어를 입력하고 클릭 한 번이면 위의 내용이 자동으로 단어장에 추가되는 기능 필요.

=> Google Gemini API 를 이용해 단어 데이터를 자동으로 생성하는 Auto-fill 기능 구현.

2. 2. 기존 단어의 빠른 검색

1. happy 라는 단어를 카드셋 A 에 추가했는데 나중에 명사형 happiness 라는 단어를 발견.

2. 이 때, 이 단어를 카드셋에 새로운 카드로 추가하기 보다는 기존 카드 happy 의 뜻에

"#명사 happiness : 행복" 이라는 내용으로 추가하고 싶음.

3. 이렇게 하려면 happy 가 포함된 카드셋 A 에 들어가서 happy 카드를 수정해야 함.

4. 하지만 cram 은 특정 단어가 어느 카드셋에 들어있는지 검색하는 기능은 제공하지 않음.

=> 전체 카드셋을 대상으로 단어 검색을 하는 기능 필요.

3. 선택한 기술 스택

3.1. Python / PySide6

본 프로젝트의 핵심 기능은 2 가지입니다.

3.1.1. 단어 뜻 자동생성 기능

단어 입력 후, 단어의 품사별 뜻, 예문, 예문 해석, 발음기호 등의 상세정보를 생성하기 위해 Gemini API 를 사용했습니다. 이 API 의 flash-lite 모델은 무료로 하루 약 100 만 토큰의 넉넉한 사용량을 제공하여 단어 검색시 유용합니다.

3.1.2. www.cram.com 처럼 단어 DB 를 온라인 상에 두고 관리하는 기능

온라인 상에서 DB 를 제공하는 클라우드 서비스는 Firebase 와 Supabase 가 있습니다. 그 중에서 저에게 친숙한 SQL 기반의 관계형 DB 를 제공하는 Supabase 를 선택했습니다. 이 사이트를 이용하여 회원가입 후, 자신의 단어 DB 를 온라인 상에 구축할 수 있도록 하였습니다.

위 2 가지 기능을 구현하기 위한 가장 성숙한 SDK 를 제공하는 Python 을 개발 언어로 선택했습니다. PySide6 는 Python 의 기본 UI 라이브러리인 Tkinter 보다 더 현대적이고 풍부한 UI 를 제공하며, QThread 를 이용해 무거운 작업을 백그라운드로 돌리고 UI 프리징을 방지할 수 있습니다.

3.2. 영단어 TTS 재생 기능 -> gTTS

3.2.1. 원어민에 가까운 고품질 음질 제공

구글 번역에서 사용하는 음성 합성 엔진을 기반으로 한 라이브러리로 원어민에 가까운 고품질의 발음을 제공합니다.

3.2.2. 사용이 간편하고 가벼운 라이브러리

구글 서버에 요청을 보내 tts 음성 파일을 생성하므로 무거운 오프라인 tts 엔진을 포함시키지 않아도 됩니다. 이 tts 음성 파일은 사용자\AppData\Local\Temp 폴더에 생성되며 네트워크 부담을 줄이기 위해 MD5 해싱을 이용한 로컬 캐싱을 구현했습니다. 사용자가 한 번 들은

단어는 텍스트를 해싱하여 tts 파일명으로 저장합니다. 동일한 tts 재생 요청시 서버에 요청하지 않고 즉시 로컬 tts 파일을 재생합니다.

4. 사용한 개발 도구

4.1. vs code / gemini cli

먼저 www.cram.com 사이트를 이용하면서 느낀 불편한 점과 개선사항을 정리하여 PRD 를 생성하였습니다. 프로젝트의 전체 뼈대와 핵심 기능을 gemini cli 를 이용해 생성한 다음, 세부 사항과 최적화 작업을 진행하였습니다.

5. 세부사항 및 최적화 작업

5.1. 100% 코드로 구현한 UI

이 프로젝트의 UI 는 Qt Designer 를 사용하지 않고 전부 코드로 구현했습니다. C# Winform, C++ MFC 로 개발해오던 경험에 비추어 처음에는 눈에 보이는 화면 없이 코드로만 구현하는 것이 낯설었지만 대부분의 작업이 레이아웃 생성 후, 위젯을 배치하는 루틴의 반복이라는 것을 깨닫고 익숙해졌습니다.

단어 DB 는 단어 카드와 단어 카드가 묶인 카드셋으로 관리됩니다. 카드셋 위젯과 단어 카드 위젯은 사용자 편집에 따라 실시간으로 생성/삭제되기 때문에 해당 위젯을 커스텀 위젯으로 구현하여 코드 상에서 정밀하게 제어할 수 있었습니다.

5.2. 사용자 친화적인 UI 로 수정

Gemini cli 가 처음에 구현한 UI 는 화면에 아예 보이지 않거나, 사용자가 기대하는 UI 반응과 거리가 멀었습니다. 예를 들어 텍스트 입력 컴포넌트의 배경색이 글자색과 동일하여 입력한 문자열이 보이지 않거나, 버튼에 마우스를 갖다 대도 변화가 없는 정적인 모습을 보여주었습니다.

5.2.1. 전역 style.css 정의

UI 가시성을 개선하고 반복된 스타일 정의 작업을 줄이기 위해 전체 프로그램에 일관적으로 적용할 스타일을 정의한 style.css 파일을 추가하였습니다.

5.2.2. 사용자 친화적인 UI 배치 및 반응 개선

사용자의 입력을 받아 사용 환경을 설정하는 부분에서 입력 컴포넌트의 순서가 난잡하게 배치되어 있던 것을 자연스러운 순서로 재배치하고 입력 기능을 개선하였습니다.

예를 들어 단어 카드를 편집할 때, 대부분의 사용자는 영단어 입력 후, tab 키를 눌러 단어 뜻을 입력하고 다시 tab 키를 눌러 새로운 단어 카드로 이동하기를 기대합니다. 이를 위해 SmartTextEdit 커스텀 컴포넌트를 정의하여 tab 키 이벤트를 처리하고, 입력한 문자열이 많으면 입력 창의 높이가 아래로 늘어나도록 개선하였습니다.

5.3. 사용자를 위한 커스터마이징 기능

5.3.1. Google API Key 설정

Gemini API 를 사용하는데 필요한 Google API Key 는 타인에게 노출되어서는 안 되는 정보입니다. 따라서 환경설정에서 자신이 사용 중인 컴퓨터 환경에 따라 key 를 2 가지 방식으로 입력할 수 있도록 하였습니다.

개인용 컴퓨터 : 입력창에 환경변수명을 넣으면 해당 환경변수에서 key 값을 읽어옵니다.

공용 컴퓨터 : 입력창에 key 값을 직접 입력합니다. 다시 로그인하면 재입력해야 합니다.

2 가지 방식 모두 입력 받은 key 값은 프로그램 메모리 내에서만 사용합니다. 제 3 자가 키를 외부로 전송하는 악의적인 코드를 추가하여 수정된 실행 파일을 배포할 수도 있기 때문에 공식 배포본에는 exe 파일의 SHA-256 해시 코드를 제공합니다.

5.3.2. AI 모델 선택

Gemini API 로 단어 상세정보를 생성할 AI 모델을 지정할 수 있도록 하였습니다.

flash-lite 모델로도 충분하지만 더 높은 생성 품질을 바란다면 고성능의 모델을 선택할 수 있습니다.

Gemini API Key 설정

영단어 입력 시 단어 뜻, 발음기호, 예문을 자동으로 생성하는 Auto-fill 기능을 사용하려면 Google AI Studio에서 발급받은 API Key가 필요합니다.

[API Key 발급받으러 가기 \(Google AI Studio\)](#)

☒ 개인용 컴퓨터 (환경변수 사용) ☐ 공용 컴퓨터 (수동 입력/위발성)

환경변수 이름 (기본값: GOOGLE_API_KEY):

GOOGLE_API_KEY

연결 테스트

사용할 AI 모델 선택:

gemini-3.1-flash-lite

Auto-fill 기능에는 경량 모델인 flash-lite나 flash 모델을 권장합니다. 더 높은 생성 품질을 원한다면 고성능 모델을 선택하세요.

위 이미지는 개인용 컴퓨터에서 환경변수로 API Key 를 입력하였습니다. 연결 테스트를 통해 사용할 AI 모델 목록을 받아올 수 있습니다. 위 예시에서는 gemini 3.1 flash-lite 모델을 선택하였습니다.

5.3.3. 단어 상세정보 생성 프롬프트의 커스터마이징

단어 정보 생성 결과를 자신의 취향대로 나오도록 프롬프트를 수정하는 기능을 제공합니다. 예를 들어 기본 제공되는 프롬프트(gemini_prompt.txt)를 사용하면 발음기호가 맨 아래에 표시되지만 이것을 맨 위에 표시되게 하고 싶다면 프롬프트를 다음과 같이 수정하면 됩니다.

gemini_prompt.txt 일부 내용

...(생략)

4. Include a #발음 section at the end with the American IPA in brackets [].

Example format for 'deliberate':

...(생략)

1. 숙고하다, 신중히 생각하다

#발음 [dɪˈlɪbəreɪt]

Return ONLY a JSON object with the key "formatted_definition" containing this text.

수정한 내용

...(생략)

4. Include a #발음 section at the **beginning** with the American IPA in brackets [].

Example format for 'deliberate':

#발음 [dɪˈlɪbəreɪt]

...(생략)

1. 숙고하다, 신중히 생각하다

Return ONLY a JSON object with the key "formatted_definition" containing this text.

5.3.4. 암기 모드에서 사용할 TTS, Star 단축키 커스터마이징 제공

단어 암기 모드에서 사용할 단축키를 지정합니다. 사용하지 않을 단축키라면 입력 창 클릭 후, Del 키를 눌러서 비활성화 처리할 수 있습니다.

학습 단축키 설정

학습 모드(단어 암기 Test)에서 사용할 단축키를 설정합니다.(Del 키를 누르면 해당 단축키를 사용하지 않습니다.)

이전 카드 (Prev):

Left

다음 카드 (Next):

Right

카드 뒤집기 (Flip):

Down

발음 듣기 (TTS):

T

중요 표시 (Star):

S

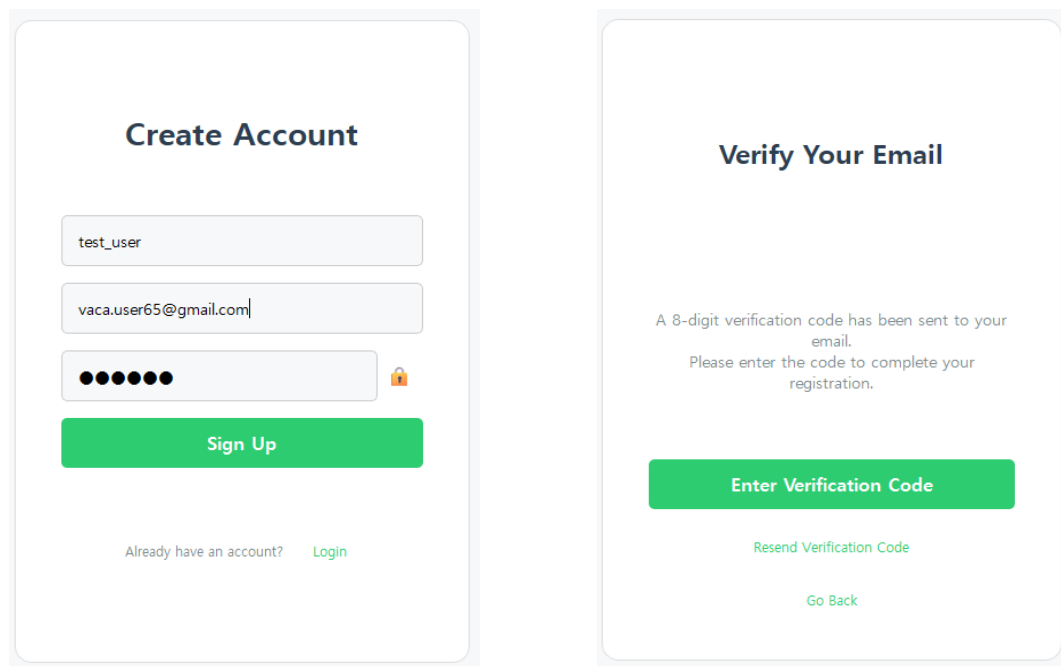
5.6. 사용자 인증 및 보안

이 프로젝트는 단어 저장소로 Supabase 를 사용하기 때문에 프로그램 사용을 위해서는 가장 먼저 Supabase DB 에 회원가입 후, 로그인해야 합니다. (여기서 회원이란 supabase 의 회원이 아니라 이 프로그램을 사용하는 회원을 뜻합니다.)

5.6.1. OTP 인증을 통한 사용자 인증

초기에는 supabase DB 에 새로운 사용자가 가입할 때, 이메일로 인증 링크를 보내는 방식이었지만 해당 링크를 열어봤더니 404 not found 화면이 뜨는 것을 확인했습니다. 이 프로젝트는 데스크톱 앱이기 때문에 웹 서버와 연동하지 않았기 때문입니다.

링크 클릭만으로 인증 자체는 이루어졌지만 이러한 경험이 사용자에게 좋지 않은 인상을 줄 것으로 생각되어 링크 대신 8 자리 OTP 인증번호를 보내어 앱에서 입력받는 방식으로 바꾸었습니다. 비밀번호 분실시에도 동일한 방식을 사용하여 인증 후, 변경할 수 있습니다.



The image displays two mobile application screens for Supabase authentication. The left screen, titled "Create Account", features three input fields: a username field containing "test_user", an email field containing "vaca.user65@gmail.com", and a password field represented by six black dots with a lock icon to its right. Below these fields is a prominent green "Sign Up" button. At the bottom, there is a link "Already have an account? Login". The right screen, titled "Verify Your Email", contains a message stating "A 8-digit verification code has been sent to your email. Please enter the code to complete your registration." Below this message is a green "Enter Verification Code" button. At the bottom of the screen, there are two links: "Resend Verification Code" and "Go Back".

<회원가입시 supabase 의 메일 서버가 가입자의 이메일로 OTP 인증 링크를 보냅니다>

5.6.2. Supabase RLS(Row Level Security)를 이용한 데이터 격리

프로젝트에 포함된 schema.sql 파일은 supabase 에 생성할 테이블 스키마를 생성하는 쿼리입니다. 이 중에 cards 테이블을 보시면 id, user_id, cardset_id 이라는 UUID 타입의 컬럼이 있습니다. 각각 해당 레코드의 id, 이 단어 카드를 생성한 유저의, id, 이 단어 카드가 속한 카드셋 레코드의 id 입니다.

UUID 는 16 바이트의 고유 문자열입니다. 같은 카드셋에 속한 카드 데이터는 유저 id, 카드셋 id 만으로 매 레코드마다 32 바이트의 중복된 데이터를 가지게 됩니다. 초기에는 이 비용이 비싸다고 생각하여 컬럼 타입을 int 로 바꾸려는 생각을 했습니다. 하지만 사용자가 자신이 생성한 카드셋, 카드 데이터만 수정할 수 있도록 하는 CURD 정책을 위해서는 불가피한 비용이라 생각하고 받아들였습니다.

5.7. 성능 최적화

5.7.1. 중복된 파일 IO 작업 제거

환경설정의 내용을 json 파일로 저장할 때, gemini cli 가 생성한 코드가 개별사항을 저장할 때마다 파일 저장을 하는 것을 발견하였습니다. 프로그램 메모리에 캐시를 생성하고 모든 개별사항을 캐시에 반영 후, 마지막에 json 파일로 저장하는 방식으로 수정하여 무거운 파일 IO 작업을 줄였습니다.

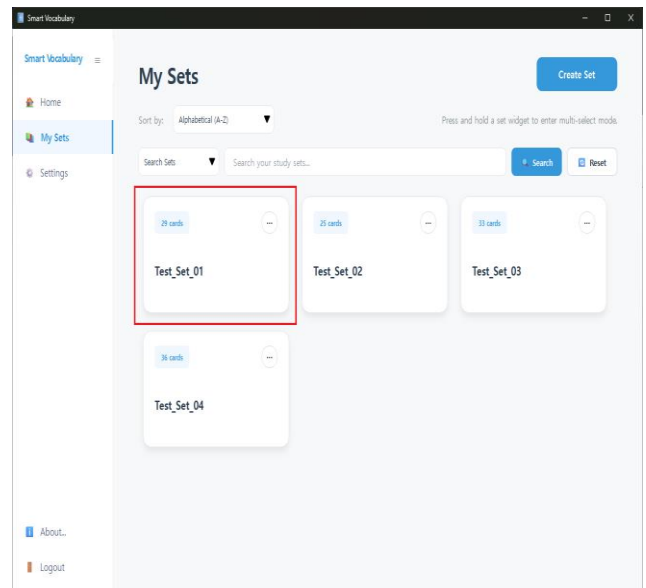
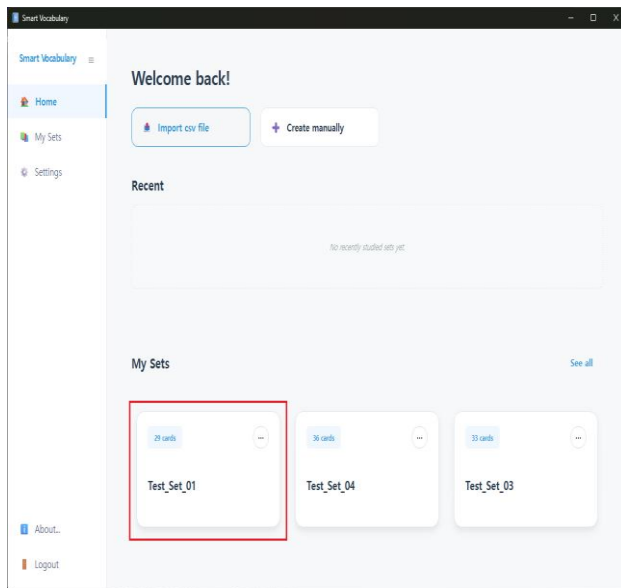
5.7.2. 네트워크 IO 작업 최적화

단어 뜻, 예문, 발음기호 등의 데이터를 생성하는 gemini api 호출은 구글 서버로부터 응답을 기다리는 무거운 작업입니다. 따라서 해당 작업을 QThread 를 사용하는 GeminiWorker 클래스로 분리하여 비동기 백그라운드 작업으로 돌렸습니다. 또한 해당 작업이 여러 번 중첩 실행되면 프로그램이 불안정해지므로 작업 중에는 화면 상에 보이는 다른 생성 버튼들은 비활성화하도록 수정하였습니다.

비동기 작업이 완료되었을 때, 생성 결과물이 UI 에 나타나기 전에는 다른 화면으로 이동하는 것을 막음으로써 비동기 작업의 결과가 처리되지 못하고 미아가 되는 상황을 방지하였습니다.

5.7.3. 코드 재사용성 강화

개인적으로 gemini cli 로 새로운 기능 구현시 비슷한 기능의 기존 코드가 있음에도 재사용을 잘 하지 않는다고 느꼈습니다. 예를 들어 좌측 사이드바 메뉴 중 Home 화면에 보이는 카드셋 위젯과 My Sets 화면에 보이는 카드셋 위젯은 기능과 모양이 동일한 커스텀 위젯입니다.



<서로 다른 화면에 보이는 위젯이지만 기능과 모양이 동일한 위젯입니다>

gemini cli 가 이 2 가지 위젯을 서로 다른 위젯 코드로 생성한 것을 동일한 코드를 사용하도록 수정하였습니다.